

How to setup OAuth 2 for your own Python script

- [Background](#)
- [Step 1 - Create project](#)
- [Step 2 - Enable API](#)
- [Step 3 - Setup oauth2 branding](#)
- [Step 4 - Prepare your credentials.json file](#)
- [Step 5 - Generate tokens](#)
- [Step 6 - Insert Oauth 2 credentials into your python script tool](#)
- [Sample Code](#)
- [References](#)

Background

You might want to do something within **Google Workspace** (Google Drive, Google Doc, Google Sheet) but Smart does not have the related tool yet. When it comes to delegation from Google, it provides various ways including:

1. Service Account
2. OAuth Delegation (OAuth 2)
3. Shared Drive

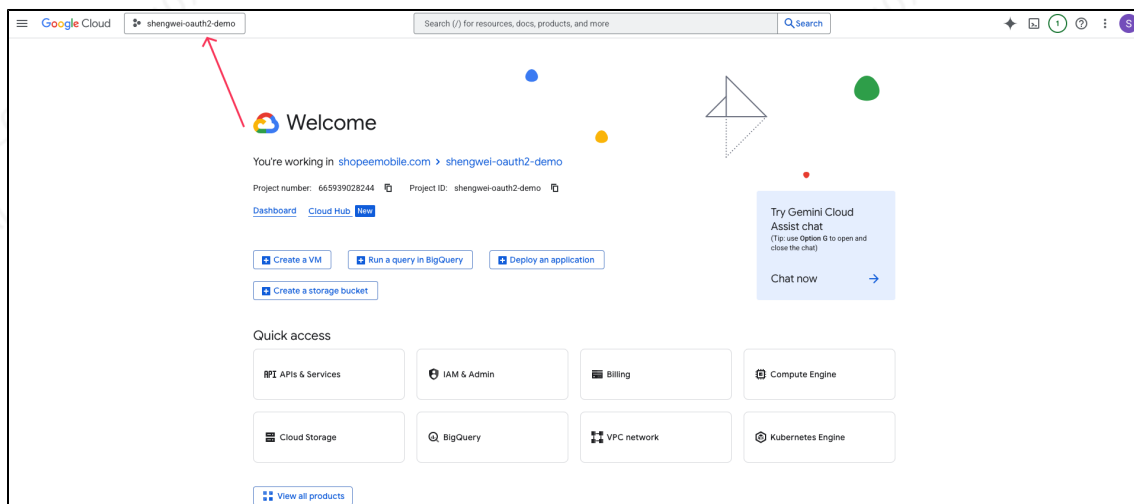
While implementing with **#1 Service account**, especially when you're trying to upload or create new files on google, you might have this error as it requires your service account to have storage quota:

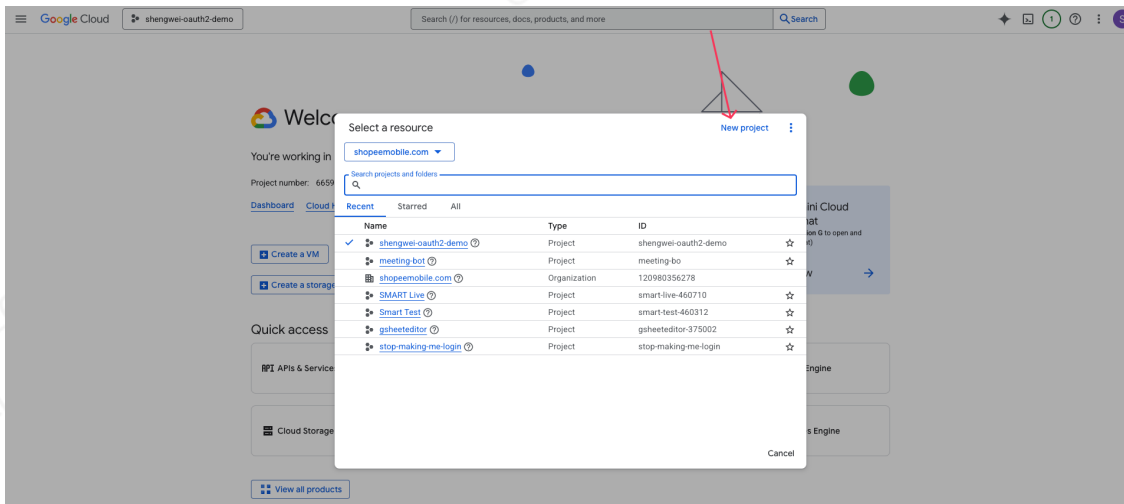
```
{"uploaded": [], "errors": [{"filename": "lixiantest.png", "error": "HTTP error while uploading: <HttpError 403 when requesting https://www.googleapis.com/upload/drive/v3/files?fields=id%2C+name%2C+mimeType&alt=json&uploadType=resumable returned \"Service Accounts do not have storage quota. Leverage shared drives (https://developers.google.com/workspace/drive/api/guides/about-shareddrives), or use OAuth delegation (http://support.google.com/a/answer/7281227) instead.\". Details: \"[{\"message\": \"Service Accounts do not have storage quota. Leverage shared drives (https://developers.google.com/workspace/drive/api/guides/about-shareddrives), or use OAuth delegation (http://support.google.com/a/answer/7281227) instead.\", \"domain\": \"usageLimits\", \"reason\": \"storageQuotaExceeded\"}]\">"}], "folder_id": "1wx6N-bt1d4tbC72zqI-ASMq0AIjFoOIk", "service_account_info_redacted": {"type": "service_account", "project_id": "named-purpose-xxxxxx-u5", "private_key_id": "3xxxxxx3c40d3ecxxxxxx", "private_key": "REDACTED", "client_email": "lixian-google-uploader@named-purpose-xxxxxx-u5.iam.gserviceaccount.com", "client_id": "11381xxxxxx47462xxxx2", "auth_uri": "https://accounts.google.com/o/oauth2/auth", "token_uri": "https://oauth2.googleapis.com/token", "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs", "client_x509_cert_url": "https://www.googleapis.com/robot/v1/metadata/x509/lixian-google-uploader%40named-purpose-xxxxxx-u5.iam.gserviceaccount.com", "universe_domain": "googleapis.com"}}
```

You can also try with **Shared Drive**, but as it might requires additional application, we will not cover this solution here. Instead, we would recommend the **OAuth 2** as the solution, which is also the focus of this playbook. Here's a step by step guide to set up the OAuth for your **python script tool**.

Step 1 - Create project

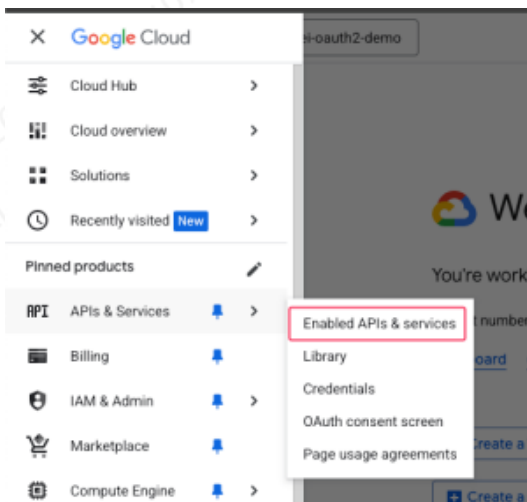
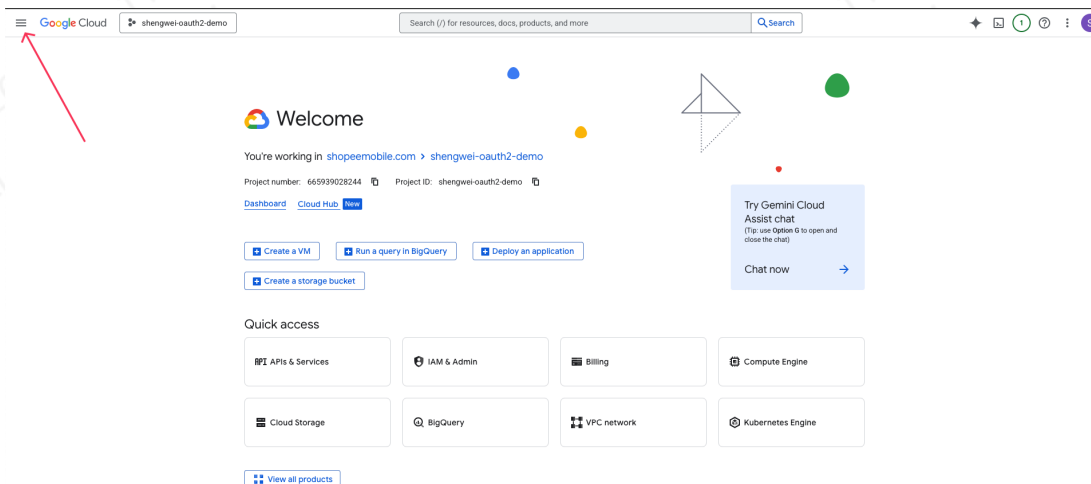
Click <https://console.cloud.google.com/>





Step 2 - Enable API

1. Click API & Services, find which service you want to enable.
2. For this example, we will choose Google Drive
3. Once you are on the product page, simply click Enable



Google Cloud

shengwei-oauth2-demo

API APIs & Services

APIs & Services

+ Enable APIs and services

Enabled APIs & services

Google Cloud shengwei-oauth2-demo

APIs & Services / API Library / Browse

API Library

google drive

API Library > "google drive"

Filter Type to filter

11 results

Visibility

Public (11)

Category

Analytics (2)

Big data (3)

Databases (2)

Maps (1)

Google Drive API

Google Enterprise API

With the Google Drive API, you can access resources from Google Drive to create files, manage file sharing, search for files and folders, and more.

Google Drive Activity API

Google Enterprise API

With the Google Drive Activity API, you can retrieve information about a user's Google Drive activity. This provides additional functionality on top of the Google Drive API for your app to display activity on a user's files, track changes to specific files or folders, or alert a user to new comments or changes to files.

Product details



Google Drive API

Google Enterprise API

Create and manage resources in Google Drive.

Enable Try this API

Step 3 - Setup oauth2 branding

Click on <https://console.cloud.google.com/auth/branding> and follow the steps

Audience - Set to internal

Contact Information - Use your own email

Step 4 - Prepare your credentials.json file

Click on <https://console.cloud.google.com/auth/clients>

Clients

+ Create client

Delete

Restore deleted OAuth clients

OAuth 2.0 Client IDs

☐	Name	Creation date ↓
No OAuth clients to display		

IMPORTANT: Choose **Desktop** app.

A client ID is used to identify a single app to Google's OAuth servers. If your app runs on multiple platforms, each will need its own client ID. See [Setting up OAuth 2.0](#) for more information. [Learn more](#) about OAuth client types.

Application type *

Desktop app

Name *

Desktop client 1

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

Note: It may take 5 minutes to a few hours for settings to take effect

Create

Cancel

IMPORTANT: Click on download JSON and store the file safely, you will not be able to get it anymore once this window is closed.

OAuth client created

The client ID can always be accessed from Clients tab under Google Auth Platform.

i OAuth access is restricted to users within your organization unless the [OAuth consent screen](#) is published and verified

Client ID 665939028244-v3o6m0bo3fr14ceg6p673b1rtvee47nc.ap
ps.googleusercontent.com

⚠ Starting in June 2025, you will no longer be able to view or download the client secret once you close this dialog. Make sure you have copied or downloaded the information below and securely stored it.

Client secret [REDACTED]

Creation date August 19, 2025 at 3:23:08 PM GMT+8

Status **✓ Enabled**

↓ Download JSON

OK

Notes: Rename the file to "**credentials.json**" otherwise you need to modify the file name in the sample code file **generate_token.py**

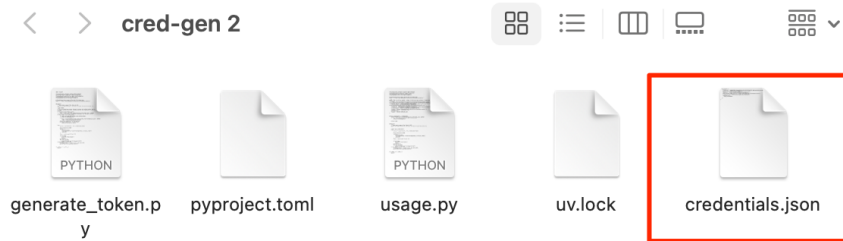
```
# time.
if os.path.exists("token.json"):
    creds = Credentials.from_authorized_user_file("token.json", SCOPES)
# If there are no (valid) credentials available, let the user log in.
if not creds or not creds.valid:
    if creds and creds.expired and creds.refresh_token:
        creds.refresh(Request())
    else:
        flow = InstalledAppFlow.from_client_secrets_file("credentials.json", SCOPES)
        creds = flow.run_local_server(port=0)
# Save the credentials for the next run
with open("token.json", "w") as token:
    token.write(creds.to_json())
```

Step 5 - Generate tokens

- Run generate_token.py
 - For your convenience, if you do not have any python environment setup, you are encouraged to use a [python package manager](#). The provided zip file already sets up the necessary dependencies

- Download the zip provided below and unzip it to a expected path.

2. Move the `credentials.json` you downloaded from Step4 to this folder



3. Open your terminal

4. Install uv:

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

5. Reload uv:

```
exec zsh # reload your shell so 'uv' is on PATH
```

6. cd to the path of the unzipped folder:

```
cd {path to folder}
```

7. Install project deps into a local .venv:

```
uv sync
```

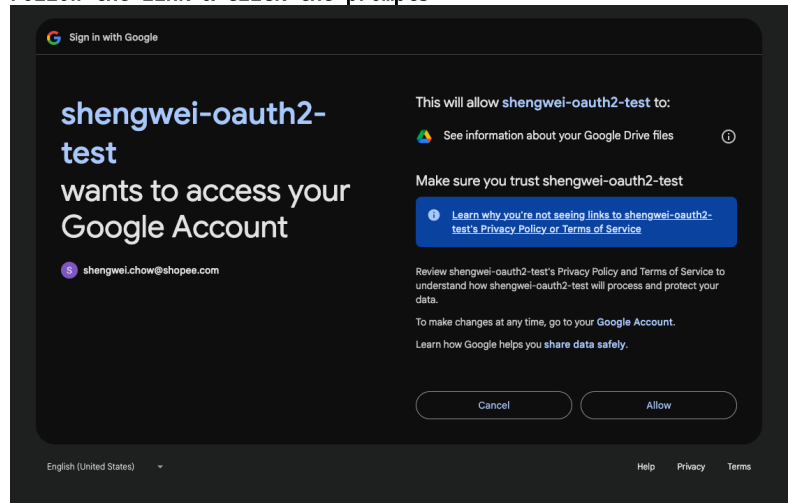
8. run the python script to generate token

```
uv run python generate_token.py
```

Please read the python file and update the "SCOPES" based on your needs. Examples:

- a. Read only: SCOPES = ["<https://www.googleapis.com/auth/drive.metadata.readonly>"]
- b. Read and write: SCOPES = ["<https://www.googleapis.com/auth/drive>"]

9. Follow the link & click the prompts



10. Once the code exe done, you'll have the token.json in your folder

1. Download the zip provided below and unzip it to a expected path.
2. Move the credentials.json you downloaded from Step4 to this folder (ensure you rename your credential file, otherwise the file cannot be fonud)
3. Open powershell
4. Install/verify uv

```
uv --version
# if not found:
winget install --id=astral-sh.uv -e
# then reopen terminal and re-run:
uv --version
```

5. Unzip and go to the project root (where pyproject.toml is)

```
cd {path}
```

6. Install dependencies into a local .venv

```
uv sync
```

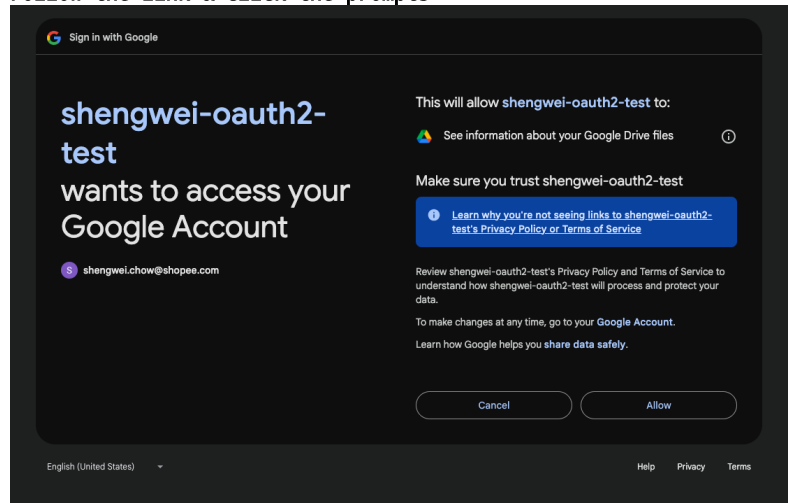
7. Run your Python script

```
uv run python generate_token.py
```

- a. Please read the python file and update the "SCOPES" based on your needs. Examples:

- i. Read only: SCOPES = ["<https://www.googleapis.com/auth/drive.metadata.readonly>"]
- ii. Read and write: SCOPES = ["<https://www.googleapis.com/auth/drive>"]

8. Follow the link & click the prompts



9. Once the code exe done, you'll have the token.json in your folder

Step 6 - Insert Oauth 2 credentials into your python script tool

Refer to usage.py in the zip for an example, of how you can use the exported token.json without a whole bunch of extra stuff. You know this flow is successful, when you see the contents of your google drive folder

Sample Code

generate_token.py

```
import os.path

from google.auth.transport.requests import Request
from google.oauth2.credentials import Credentials
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build
from googleapiclient.errors import HttpError

# If modifying these scopes, delete the file token.json.
SCOPES = ["https://www.googleapis.com/auth/drive.metadata.readonly"]

def main():
    """Shows basic usage of the Drive v3 API.
    Prints the names and ids of the first 10 files the user has access to.
    """
    creds = None
    # The file token.json stores the user's access and refresh tokens, and is
    # created automatically when the authorization flow completes for the first
    # time.
    if os.path.exists("token.json"):
        creds = Credentials.from_authorized_user_file("token.json", SCOPES)
    # If there are no (valid) credentials available, let the user log in.
    if not creds or not creds.valid:
        if creds and creds.expired and creds.refresh_token:
            creds.refresh(Request())
        else:
            flow = InstalledAppFlow.from_client_secrets_file("credentials.json", SCOPES)
            creds = flow.run_local_server(port=0)
        # Save the credentials for the next run
        with open("token.json", "w") as token:
            token.write(creds.to_json())

    try:
        service = build("drive", "v3", credentials=creds)
```



```
# Call the Drive v3 API
results = (
    service.files()
        .list(pageSize=10, fields="nextPageToken, files(id, name)")
        .execute()
)
items = results.get("files", [])

if not items:
    print("No files found.")
    return
print("Files:")
for item in items:
    print(f'{item["name"]} ({item["id"]})')
except HttpError as error:
    # TODO(developer) - Handle errors from drive API.
    print(f"An error occurred: {error}")

if __name__ == "__main__":
    main()
```



generate_token.zip

References

- https://developers.google.com/workspace/drive/api/quickstart/python#configure_the_oauth_consent_screen